**Análisis de algoritmos y notación Big O**

Entre un programa de 1.000 operaciones y otro de 1.000.000 para el mismo problema, elegiría el de 1.000 operaciones: con millones de registros, la diferencia de trabajo se vuelve enorme y el segundo sería inviable.

## 1. Clasificación de los cinco fragmentos

| Fragmento | Operaciones aprox. | Big O | Justificación |
|---|---|---|---|
| Acceder a vector[5] | 1 | O(1) | Acceso directo por índice; no depende del tamaño del arreglo. |
| Recorrer un arreglo | n | O(n) | Un ciclo visita cada elemento una sola vez. |
| Dos ciclos anidados | $n \times n = n^2$ | $O(n^2)$ | Por cada vuelta del ciclo externo, el interno da n vueltas. |
| Dividir n entre 2 repetidamente | $\log_2 n$ | O(log n) | El problema se reduce a la mitad en cada paso. |
| Ciclo n + reducción logarítmica | $n \cdot \log n$ | O(n log n) | El ciclo externo hace n vueltas y en cada una el interno hace log n. |

Reto adicional: no es $O(n^2)$. Los dos ciclos son consecutivos, no están anidados, así que las operaciones se suman: $n + n = 2n$. Big O descarta las constantes, por lo que la complejidad es O(n).

## 2. Tabla de tiempos

| Tamaño n | Tiempo O(n) (ns) | Tiempo $O(n^2)$ (ns) | ¿Cuál fue más rápido? |
|---|---|---|---|
| 100 | 5.700 | 133.600 | O(n) |
| 1.000 | 11.200 | 2.312.300 | O(n) |
| 5.000 | 40.300 | 10.244.900 | O(n) |
| 10.000 | 82.000 | 5.500 | $O(n^2)$ * |

Ejecución 1 del programa ComparacionAlgoritmos.java, en nanosegundos, obtenidos con System.nanoTime(). Dependen del computador y cambian entre ejecuciones. * El tiempo de $O(n^2)$ con n = 10.000 es atípico y no refleja la complejidad real (ver pregunta 8).

**Variación entre dos ejecuciones (tiempo O(n) / tiempo $O(n^2)$, en ns)**

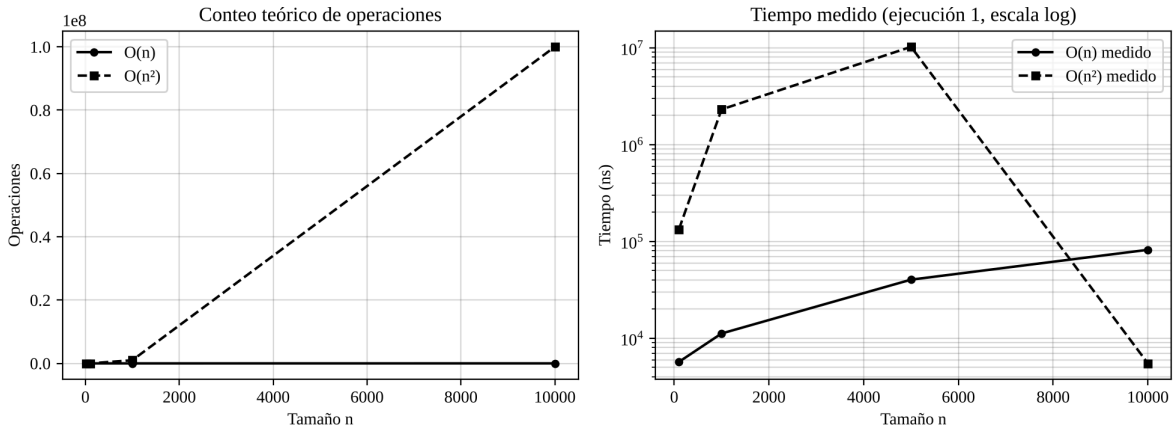
| Ejecución | n = 100 | n = 1.000 | n = 5.000 | n = 10.000 |
|---|---|---|---|---|
| 1 | 5.700 / 133.600 | 11.200 / 2.312.300 | 40.300 / 10.244.900 | 82.000 / 5.500 |
| 2 | 8.800 / 162.600 | 24.700 / 3.210.200 | 50.400 / 13.952.800 | 159.500 / 7.400 |

**Conteo teórico (Parte D)**

| n | O(1) | O(log n) aprox. | O(n) | O(n log n) aprox. | O(n²) |
|---|---|---|---|---|---|
| 10 | 1 | 3 | 10 | 33 | 100 |
| 100 | 1 | 7 | 100 | 664 | 10.000 |
| 1.000 | 1 | 10 | 1.000 | 9.966 | 1.000.000 |
| 10.000 | 1 | 13 | 10.000 | 132.877 | 100.000.000 |

Logaritmos en base 2, redondeados al entero más cercano.

## 3. Gráfica comparativa




A la izquierda, el conteo teórico: la línea de O(n) casi no se separa del eje mientras O(n²) se dispara. A la derecha, los tiempos medidos en escala logarítmica, donde se ve que el algoritmo cuadrático queda por encima del lineal hasta n = 5.000 y la distancia entre ambos crece con n. El punto del cuadrático en n = 10.000 baja de forma atípica; la causa se explica en la pregunta 8.

## 4. Respuestas a las preguntas de análisis

**1. ¿Qué diferencia existe entre tiempo de ejecución y complejidad Big O?**

El tiempo de ejecución es lo que realmente mide el computador (segundos o nanosegundos) y depende del procesador, la memoria, el sistema operativo y hasta de la JVM. Big O, en cambio, no habla de segundos: describe cómo crece el número de operaciones cuando crece n, sin importar la máquina.

**2. ¿Por qué O(1) se considera una complejidad eficiente?**

Porque el trabajo no cambia aunque los datos crezcan. Consultar vector[5] cuesta lo mismo con 10 elementos que con 1.000.000.

**3. ¿Qué ocurre con un algoritmo O(n) cuando duplicamos el tamaño de entrada?**

El trabajo también se duplica, porque crece de forma proporcional a n.

**4. ¿Qué ocurre aproximadamente con un algoritmo $O(n^2)$ cuando duplicamos n?**
El trabajo se multiplica por cuatro, ya que $(2n)^2 = 4n^2$.

**5. ¿Por qué los ciclos anidados suelen relacionarse con $O(n^2)$?**
Porque por cada vuelta del ciclo externo (n vueltas) el ciclo interno vuelve a dar n vueltas completas, así que las operaciones se multiplican: $n \times n$.

**6. ¿Qué característica permite reconocer un algoritmo O(log n)?**
Que en cada paso el problema se reduce a una fracción (por ejemplo, a la mitad), como en el ejercicio donde $n = n / 2$. No se recorre elemento por elemento, se va descartando mucho de golpe.

**7. ¿Cuál de los dos algoritmos experimentales presentó mayor crecimiento?**
El cuadrático. En la ejecución 1, al pasar de n = 100 a n = 5.000 (50 veces más datos), el lineal subió de 5.700 a 40.300 ns (unas 7 veces) y el cuadrático de 133.600 a 10.244.900 ns (unas 77 veces). Con n = 100 los valores son poco confiables por el arranque de la JVM, y el dato de n = 10.000 del cuadrático no sirve para comparar por la razón que explico en la pregunta 8, pero la tendencia es clara.

**8. ¿Los resultados medidos fueron exactamente iguales entre las diferentes ejecuciones? ¿Por qué?**
No. Ejecuté el programa dos veces y ningún tiempo se repitió (ver tabla de variación). Cambian por el procesador y la caché, otros procesos del sistema operativo y la compilación JIT de Java. De hecho, en las dos ejecuciones el algoritmo cuadrático con n = 10.000 salió en apenas 5.500 y 7.400 ns, mucho menos que con n = 1.000, algo imposible para un $O(n^2)$. Lo más probable es que la JVM optimizó ese ciclo porque su resultado no se usa. Por eso no se debe confundir tiempo medido con complejidad Big O.

**9. Si tuviera que procesar diez millones de registros, ¿qué tipo de complejidad preferiría?**
La más baja posible: O(1) o O(log n) si el problema lo permite y, si hay que tocar todos los datos, O(n) u O(n log n). Un $O(n^2)$ significaría unos $10^{14}$ operaciones; a razón de mil millones por segundo serían más de 27 horas.

**10. ¿Por qué un ingeniero de software debe analizar la eficiencia antes de seleccionar un algoritmo?**
Porque funcionar con pocos datos no garantiza funcionar con muchos. Elegir mal sale caro: aplicaciones lentas, más recursos de servidor y costos, y corregirlo después es mucho más difícil que decidirlo bien desde el diseño.

## 5. Conclusión personal

Antes de esta actividad pensaba que si un programa daba el resultado correcto, ya estaba bien. El experimento me mostró que no es tan simple: con n = 100 los dos tiempos eran minúsculos, pero con n = 5.000 el cuadrático tardó alrededor de 10 milisegundos frente a 0,04 milisegundos del

lineal. Lo que más me llamó la atención fue que los tiempos cambian de una ejecución a otra incluso con el mismo código, y que con n = 10.000 el cuadrático dio un tiempo menor que con n = 1.000, así que medir segundos no reemplaza entender la complejidad. Big O no me dice cuánto va a demorar un programa, me dice cómo va a crecer. De ahora en adelante, antes de elegir un algoritmo me voy a preguntar qué pasa si los datos se multiplican por mil, porque una aplicación con millones de registros no perdona un O(n²).

